

Lab 8 - Leader-Based Replication and Majority Commit

Lab Overview

Implement a simplified replication system inspired by leader-based consensus protocols such as Raft.

Solution Overview

One node acts as Leader. Other nodes act as Followers.

Message types:

- REPLICATE increment
- ACK
- COMMIT

Each node stores a counter. When client sends (increment), the leader tentatively increments its counter (counter = 5; tentative), then sends (REPLICATE 5) to all followers.

When the follower receives (REPLICATE 5), it should update its local value (counter = 5) and send (ACK) back to the leader.

The leader counts the received ACKs and commits only if acks $\geq$ majority (For 5 nodes, majority = 3 nodes = leader + at least 2 followers). Then the leader prints (COMMIT SUCCESS). Otherwise, it prints (COMMIT FAILED).

Follower

Each follower:

- listens on a port
- receives replication requests
- updates counter
- sends ACK

```
public class Follower {
    private int counter = 0;
    public Follower(int port) throws Exception {
        ServerSocket server = new ServerSocket(port);
        System.out.println("Follower started on port " + port);
    }
}
```

```

        while(true){
            Socket socket = server.accept();
            BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter out = new PrintWriter(socket.getOutputStream(),true);
            String msg = in.readLine();
            if(msg.startsWith("REPLICATE")){
                int value = Integer.parseInt(msg.split(" ")[1]);
                counter = value;
                System.out.println("Updated counter = " + counter);
                out.println("ACK");
            }
            socket.close();
        }
    }
    public static void main(String[] args) throws Exception {
        int port = Integer.parseInt(args[0]);
        new Follower(port);
    }
}

```

Leader

```

public class Leader {
    private int counter = 0;
    private static final String[] HOSTS = {
        "localhost","localhost",
        "localhost","localhost"
    };

    private static final int[] PORTS = {5001,5002,5003,5004};

    public void processRequest() {
        counter++;
        System.out.println("\n Tentative value = " + counter);
        int ackCount = 1; // leader counts itself
        for(int i=0;i<PORTS.length;i++){
            try{
                Socket socket = new Socket(HOSTS[i],PORTS[i]);
                PrintWriter out = new
PrintWriter(socket.getOutputStream(),true);

```

```

        BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        out.println("REPLICATE " + counter);
        String reply = in.readLine();
        if(reply.equals("ACK")){
            ackCount++;
            System.out.println("ACK from follower "+(i+1));
        }
        socket.close();
    }
    catch(Exception e){
        System.out.println("Follower "+(i+1)+" unavailable");
    }
}
if(ackCount >= 3)
    System.out.println("COMMIT SUCCESS");
else
    System.out.println("COMMIT FAILED (no majority)");
}

public static void main(String[] args) throws Exception {
    Leader leader = new Leader();
    ServerSocket server = new ServerSocket(6000);
    System.out.println("Leader listening on 6000");
    while(true){
        Socket socket = server.accept();
        BufferedReader in = new BufferedReader( new
InputStreamReader(socket.getInputStream()));
        String req = in.readLine();
        if(req.equals("increment"))
            leader.processRequest();
        socket.close();
    }
}
}

```

Client

```

public class Client {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 6000);
        PrintWriter out = new PrintWriter( socket.getOutputStream(),true);
        out.println("increment");
        socket.close();}}

```

Execution

Run Followers (4 terminals)

Run Leader

Run Client (several times)

Kill 3 Followers and Run Client again.

Simulate slow nodes by adding `Thread.sleep(3000)` inside Follower before ACK.

Lab Extension (to be done)

Add a Replicated Log

Instead of storing only one counter, nodes store a **log of operations**. This brings the lab much closer to Raft without implementing full Raft.